LESSON 03 · WEBSOCKET · 50 MIN

# 3교시 실시간 — 왜 새로고침을 해야 최신이 보일까

화면에 주문 수가 떠 있는데 다른 사람이 주문해도 내 화면은 그대로입니다. 이 문제를 푸는 방법이 세 가지 있고 각각 대가가 다릅니다. 셋을 다 해보고 차이를 몸으로 느껴 봅니다.

설명 10분 · 따라하기 30분 · 정리와 질문 10분

OVERVIEW

## 최신 정보를 받는 세 가지 방법

**① 내가 새로고침**
누를 때만 갱신

자동화하면 →

**② 계속 물어보기**
2초마다 자동

더 나아가면 →

**③ 연결 유지**
한 번 열고 기다림

②는 편해지지만 **대부분 헛수고**입니다. “새 소식 있어?” “없어” 를 계속 반복하니까요. ③은 선을 열어두고 기다리므로 요청은 한 번인데 소식은 즉시 옵니다.

WORDS

## 새로 나오는 말

| 말 | 무슨 뜻인가 | 언제 나오나 |
|---|---|---|
| 새로고침 | 내가 직접 다시 물어보기 | 가장 단순 |
| 폴링 | 일정 간격으로 자동 반복 | 편하지만 낭비가 많다 |
| 연결 유지 | 선을 열어두고 기다리기 | 요청 1번 · 즉시 도착 |
| WebSocket | 연결을 유지하는 대표 방법 | 양쪽이 서로 보낼 수 있다 |
| SSE | 서버만 보내는 간단한 방법 | 한 방향이면 이걸로 충분 |
| 101 | 연결 방식이 바뀌었다는 응답 | WebSocket 이 열릴 때 나온다 |
| 연결 끊김 | 선이 조용히 죽는 것 | 화면에는 표시가 안 난다 |
| 재연결 | 끊기면 다시 붙기 | 직접 만들어야 한다 |

FILES

## 실습용 파일 — 그대로 옮겨 적으세요

ws_server.py

```
import asyncio, itertools
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()
CLIENTS = set()
counter = itertools.count(1)


async def producer():          # 3초마다 새 주문이 들어온다고 가정
    while True:
        await asyncio.sleep(3)
        n = next(counter)
        for ws in list(CLIENTS):
            try:
                await ws.send_text(f"새 주문! 지금까지 {n}건")
            except Exception:
                CLIENTS.discard(ws)


@app.on_event("startup")
async def start():
    asyncio.create_task(producer())


@app.websocket("/ws")
async def ws_endpoint(ws: WebSocket):
    await ws.accept()
    CLIENTS.add(ws)
    try:
        while True:
            await ws.receive_text()      # 그냥 연결만 유지
    except WebSocketDisconnect:
        CLIENTS.discard(ws)
```

STEPS

## 실습 순서

## STEP 01 새로고침 문제 느껴 보기

화면이 왜 낡는지 확인합니다.

```
# 2교시 서버를 켜 둔 채로
# 브라우저 창을 두 개 엽니다

# 창 A: http://127.0.0.1:8000/orders  (목록 보기)
# 창 B: http://127.0.0.1:8000/docs   (POST 로 주문 추가)

# 창 B에서 주문을 하나 추가하고 창 A를 그대로 두세요
```

**확인 포인트**

- [ ] 창 A의 내용이 **바뀌지 않는다**
- [ ] 창 A를 새로고침해야 새 주문이 보인다

> 이게 문제입니다. **서버는 바뀌었는데 내 화면은 모릅니다.** 카톡이 새로고침 없이 뜨는 걸 생각하면, 뭔가 다른 방법이 있다는 걸 알 수 있죠.

## STEP 02 계속 물어보게 해 보기 (폴링)

자동으로 물어보면 어떻게 되는지 봅니다.

```
# 창 A에서 F12 → Console 탭에 아래를 붙여넣고 엔터
let n = 0;
const t = setInterval(async () => {
  const r = await fetch('http://127.0.0.1:8000/orders');
  const list = await r.json();
  console.log(++n, '번째 물어봄 →', list.length, '건');
}, 2000);

# 멈추려면
clearInterval(t);
```

**확인 포인트**

- [ ] 2초마다 한 줄씩 찍힌다
- [ ] **대부분 지난번과 똑같은 숫자** 다 — 헛수고
- [ ] 창 B에서 주문을 추가하면 다음 줄부터 숫자가 올라간다

> 편해졌지만 **대부분 헛수고**입니다. 간격을 1초로 줄이면 더 빨리 알지만 헛수고도 두 배가 됩니다.
> **빨리 아는 것과 덜 낭비하는 것을 동시에 얻을 수는 없습니다.**

## STEP 03 WebSocket 서버 준비하기

연결을 유지하는 서버를 띄웁니다.

```
pip install "uvicorn[standard]"

# 위 '실습용 파일'의 ws_server.py 를 만듭니다
# (그대로 붙여넣으세요 — 지금은 이해 안 해도 괜찮습니다)

# 새 터미널에서 (2교시 서버는 켜 둔 채로)
uvicorn ws_server:app --reload --port 8001
```

**확인 포인트**

- [ ] 8001 포트로 서버가 하나 더 뜬다
- [ ] 터미널이 두 개 열려 있고 둘 다 돌고 있다

> **uvicorn[standard]** 를 꼭 설치하세요. 이게 없으면 연결이 거부됩니다.
> 코드를 이해하려 애쓰지 마세요. 오늘은 **동작을 보는 것**이 목표입니다.

## STEP 04 브라우저에서 연결하기

연결이 열리는 순간을 봅니다.

```
// 브라우저 Console 에 붙여넣기
const ws = new WebSocket('ws://127.0.0.1:8001/ws');
ws.onopen = () => console.log('연결 열림!');
ws.onmessage = e => console.log('받음:', e.data);
ws.onclose = () => console.log('연결 닫힘');
```

확인 포인트

- [ ] **연결 열림!** 이 찍힌다
- [ ] 3초마다 **새 주문!** 메시지가 저절로 온다
- [ ] 내가 아무것도 안 했는데 계속 온다

> 여기가 오늘의 핵심 장면입니다. **요청은 한 번뿐인데 메시지는 계속 옵니다.**
> 폴링에서는 메시지 하나당 요청 하나가 필요했죠. 이제 요청 없이 받습니다.

## STEP 05 101 확인하기

연결이 어떻게 시작됐는지 봅니다.

```
# F12 → Network 탭 → 위쪽 필터에서 WS 클릭
# ws 라는 줄을 클릭해 보세요
```

확인 포인트

- [ ] Status 칸에 **101** 이라고 적혀 있다
- [ ] Messages 탭에서 오간 메시지 목록이 보인다

> 1교시에 본 200, 404 같은 숫자죠. **101은 "이제부터 방식을 바꾸자"** 는 뜻입니다.
> 이 순간부터는 요청/응답이 아니라 **열린 선**이 됩니다.

## STEP 06 끊어 보기 — 조용히 죽는다

연결 유지 방식의 약점을 봅니다.

```
# 8001 서버 터미널에서 Ctrl+C 로 강제 종료

// Console 에서 상태 확인
ws.readyState    // 1이면 아직 열린 줄 안다

// 잠시 뒤 다시 확인
ws.readyState    // 3이면 닫힘
```

확인 포인트

- [ ] 메시지가 더 이상 오지 않는다
- [ ] 그런데 **화면에는 아무 경고도 안 뜬다**
- [ ] readyState 가 한동안 1로 남아 있기도 하다

> **이게 가장 중요한 교훈입니다.** 연결이 끊겨도 화면은 마지막 숫자를 계속 보여줍니다. 사용자는 최신인 줄 알죠.
> 그래서 실제로 만들 때는 ① 끊긴 걸 알아채고 ② 다시 연결하고 ③ 놓친 걸 채우는 일을 **직접** 해야 합니다. 새로고침 방식에는 없던 숙제예요.

## STEP 07 다시 연결하게 만들기

가장 간단한 재연결을 붙여 봅니다.

```
// 서버를 다시 켠 뒤, Console 에 붙여넣기
let delay = 1000;
function connect() {
  const ws = new WebSocket('ws://127.0.0.1:8001/ws');
  ws.onopen = () => { console.log('연결됨'); delay = 1000; };
  ws.onmessage = e => console.log('받음:', e.data);
  ws.onclose = () => {
    console.log(delay/1000, '초 뒤 다시 시도');
    setTimeout(connect, delay);
    delay = Math.min(delay * 2, 30000);
  };
}
connect();

# 서버를 껐다 켜 보세요
```

확인 포인트

- [ ] 서버를 끄면 "1초 뒤 다시 시도" 가 찍힌다
- [ ] 계속 실패하면 2초, 4초로 **간격이 늘어난다**
- [ ] 서버를 다시 켜면 스스로 붙는다

> 간격을 늘리는 이유는 **죽은 서버를 계속 두드리지 않기 위해서**입니다.
> 다만 끊긴 동안 온 메시지는 여전히 사라집니다. 그것까지 채우려면 코드가 더 필요해요 — 실시간이 공짜가 아닌 이유입니다.

## STEP 08 언제 무엇을 쓸지 정리

판단 기준을 세웁니다.

```
# 가끔 확인해도 되는 정보
→ 새로고침이나 폴링으로 충분

# 서버에서 화면으로만 계속 보내는 경우 (알림, 진행률)
→ SSE 가 더 간단

# 양쪽이 서로 주고받아야 하는 경우 (채팅, 게임)
→ WebSocket
```

확인 포인트

- [ ] 세 경우를 구분해 말할 수 있다

> **실시간이라는 말만 듣고 반사적으로 WebSocket 을 고르지 마세요.** 고르는 순간 재연결과 끊김 처리라는 숙제가 따라옵니다.

WHEN STUCK

## 막혔을 때 보는 곳

| 증상 | 원인 | 해결 |
|---|---|---|
| WebSocket 연결이 바로 닫힘 | uvicorn[standard] 미설치 | pip install "uvicorn[standard]" 후 서버 재시작 |
| Console 에 붙여넣었는데 오류 | 앞의 // 주석까지 복사함 | 코드 부분만 복사 · 한 덩어리씩 나눠 붙여넣기 |
| 메시지가 안 옴 | 서버 포트가 다름 | 주소의 8001 이 맞는지 확인 · 터미널에 뜬 포트와 비교 |
| Network 탭에 WS 필터가 없음 | 연결 전에 탭을 염 | 탭을 켜 둔 채로 연결 코드를 다시 실행 |
| 회사 네트워크에서만 안 됨 | 보안 장비가 막음 | 127.0.0.1 로 실습 · 사내망 이슈는 강사에게 알리기 |

CHECK

## 스스로 확인하기 — 답할 수 있으면 통과

- [ ] 왜 새로고침을 해야 최신이 보였나요?
- [ ] 폴링의 좋은 점과 나쁜 점을 하나씩 말해 보세요.
- [ ] 폴링 간격을 줄이면 무엇이 좋아지고 무엇이 나빠지나요?
- [ ] 101 이라는 숫자는 무슨 뜻인가요?
- [ ] 연결이 끊겼을 때 왜 위험한가요?

한 줄 정리

연결을 유지하면 빠르지만, 끊김을 챙기는 일은 온전히 내 몫이 됩니다.

다음 시간 → 4교시 — 여기서부터는 AI 이야기입니다

**WORKSHEET**

막힌 STEP 번호 · 화면에 나온 빨간 글씨 · 강사에게 물어볼 것